7-Day Study Plan

If your timeline extends

Prep materials — Anthropic Fellows OA

August 2026

Contents

7-Day Prep Plan — Anthropic Fellows OA

Assessment in 7 days. Format (per candidate reports): ~90 min, one system simulated across escalating levels (600 pts), pre-built classes + README, domain = LLM inference engine (prefill/decode, KV cache, load balancing). The algorithms are easy; the bottleneck is decoding an unfamiliar codebase under time pressure. Prep weights reflect that.

Daily time: ~2–3 hours. Each day = one systems block + one CP block.

Day 1 — Domain foundations

- Read `inference_engine_primer.md` fully (~45 min). Redraw the prefill/ decode timeline and the continuous-batching tick loop from memory.
- Watch/read one external explainer on vLLM continuous batching + KV cache.
- CP (45 min): warm-up simulation problems — implement a FIFO event loop, a min-heap task scheduler. Python only (that's the OA language reported).

Day 2 — Build it yourself

- From scratch, no references: write a tiny discrete-tick simulator — N requests, 1 worker, prefill then decode, FIFO. Then add a memory limit. (~60 min. This is the single highest-value exercise this week.)
- CP (45 min): heap + queue problems (e.g. "process tasks with cooldown", "CPU scheduling" style LeetCode: 621, 1834, 2402).

Day 3 — Codebase-reading drills

- Pick an unfamiliar mid-size Python repo (or a class-heavy Gist) and give yourself 15 min to answer: what are the entry points, what do the core classes own, where would I add feature X? Repeat with a second repo.
- Skim vLLM's actual `scheduler.py` on GitHub for 30 min — don't study it, just practice extracting structure fast.
- CP (45 min): sorting with custom keys, dict/counter problems. Practice writing deterministic tie-breaks quickly: `key=lambda x: (a, b, id)`.

Day 4 — MOCK OA (timed)

- 90 minutes, strict, no AI, no search beyond Python docs: do `mock_oa/` (`python tests.py` to score). Treat it exactly like the real thing.
- Afterwards read `SPOILERS.md` and debrief: where did the time go?
- No CP today.

Day 5 — Patch the holes

- Whatever level broke on Day 4, rebuild that component from scratch and re-derive the tick semantics on paper.
- Drill the OA meta-skills: (a) read tests first, (b) get level 1 green in ≤20 min, (c) commit/submit early and often if the platform allows.
- CP (60 min): 2 medium simulation problems, timed 25 min each.

Day 6 — Second mock + breadth

- Re-run the mock with a twist: restore the stubs, then implement Level 2 with a *different* policy (shortest-job-first admission) and predict the outputs before running — checks you truly own the tick model.
- Skim primer sections 5–7 again (scheduling policies, metrics) — likely level-3/4 material.
- CP (30 min): light — one easy, one medium.

Day 7 — Taper

- 30 min: reread primer vocabulary + your Day 4 debrief notes.
- No new material, no hard problems. Sleep well — the OP of that Reddit thread lost the test to nerves as much as anything.

Test-day protocol

1. First 10–15 min: read README **and the visible tests/examples** before writing code. Note every input/output format and tie-break rule.
2. Trust tests > code signatures > README when they conflict (they will).
3. Level 1 fast, then run the grader — a green level 1 both scores points and validates your understanding of the tick model before you build on it.
4. Budget: ~15/20/25/30 min per level; abandon a stuck level, bank partial credit. Reported cutoffs sit around 60% of max, so clearing levels 1–3 cleanly is the realistic target — don't sink the clock into level 4.
5. Expect a later level to **invalidate an earlier assumption** (the mock's level 4 drops head-of-line blocking). Keep policies swappable; don't hard-code admission logic deep inside the tick loop.
6. If panic hits: 60 seconds of slow breathing, then restate the current level's contract in one written sentence before touching code.

What NOT to over-prep

- Hard CP (graphs, DP, advanced algorithms) — no evidence it appears. Queues, heaps, dicts, sorting, and careful simulation cover it.
- GPU/ML math — the OA models systems behavior, not attention math.